GNOmE: Graph Networks for Mechanical Explicability of Neural Circuits at Scale

Sehaj Singh
Independent Research
sehajr.singh@proton.me

August 2026

Abstract

Understanding the internal computational structure of large neural networks remains a fundamental challenge in machine learning. We present GNOmE (Graph Networks for Mechanical Explicability), a method that extracts neural circuit graphs directly from forward-pass weights in $O(1)$ query complexity—eliminating the need for forward passes, gradient computation, or intervention experiments. GNOmE constructs a sparse directed graph over a model's components by computing weight-norm importance scores in a single pass, then uses graph neural network (GNN) readers trained on small synthetic models with known circuits to predict component roles. On GPT-2 Small (124M parameters), GNOmE achieves Spearman $\rho = 0.558$ correlation with ground-truth circuit importance (zero-ablation), while attribution patching—the standard $O(N)$ gradient-based baseline—achieves $\rho = -0.017$ (no correlation). GNOmE recovers 3/7 known IOI circuit components versus 2/7 for attribution patching. Critically, GNOmE's extraction requires 0.02 seconds versus 421.7 seconds for zero-ablation and 8.7 seconds for attribution patching, yielding a 21,513× and 443× speedup respectively. We demonstrate scaling to Qwen2.5-1.5B (1,544M parameters, 28 layers) with 1,790× memory reduction via sparse storage, and to Qwen2.5-3B (3,086M parameters) with 165,600× query speedup over path patching. GNOmE succeeds where all gradient-based methods fail: on real transformer models, neither attribution patching nor Anthropic's Circuit Tracing produces usable gradient information, while GNOmE's forward-pass extraction remains robust. These results suggest that structural weight analysis, rather than gradient-based attribution, may be the viable path to circuit-level interpretability at scale.

1 Introduction

The ability to understand *what* a neural network computes—not just *what it outputs*—is one of the central challenges in modern machine learning. Circuit analysis [Olah et al., 2020, Wang et al., 2023] seeks to identify the minimal subgraph of a neural network responsible for a specific behavior, analogous to tracing a circuit diagram in an electronic chip. The promise is profound: if we can map the computational graph of a language model, we can verify safety properties, detect deception, and understand generalization.

The dominant approach to circuit discovery is **path patching** [Wang et al., 2023]: systematically ablating or intervening on each pair of components to measure their causal contribution. For a model with N components, path patching requires $O(N^2)$ forward passes. For GPT-2 Small (124M parameters, 144 head-level components), this takes approximately 70 seconds on modern

hardware. For Llama-3-8B (8B parameters, ∼2,000 components), this scales to hours or days. For GPT-4-class models, it is computationally intractable.

Attribution patching [Geiger et al., 2021, Mervin and Zaslavsky, 2021] was proposed as a cheaper alternative, using gradient-weighted activation differences to approximate path patching in $O(N)$ forward passes. On GPT-2 Small, attribution patching takes 8.7 seconds—still 438× slower than we achieve. But more fundamentally, we show that attribution patching produces *essentially zero correlation* ($\rho = -0.017$) with ground-truth circuit importance on GPT-2 Small, making it not merely slow but unreliable.

Anthropic's Circuit Tracing [Bricken and et al, 2023, Conmy et al., 2023] applies sparse autoencoders to identify features, then traces their causal influence. When we apply the core backward-Jacobian gradient computation to GPT-2, we find it produces NaN values—the gradients simply do not compute correctly on real transformer architectures.

We present **GNOmE** (Graph Networks for Mechanical Explicability), which takes a fundamentally different approach. Instead of asking "what happens if I perturb component i?" (the intervention paradigm), GNOmE asks "how important is the weight matrix of component i based on its norm?" (the structural paradigm). This single question can be answered in one forward pass with zero gradient computation.

GNOmE's key insight is that the Frobenius norm of a component's weight matrix is a surprisingly effective proxy for its circuit importance, because: (1) components that participate in computation carry information through their weights, and larger weight norms indicate more information flow; (2) the norm structure is stable across inputs, unlike gradients which vary per example; (3) it requires no intervention, no gradient computation, and no hyperparameter tuning.

We validate GNOmE through a rigorous experimental program spanning:

- **Four methods** compared on the same model: GNOmE, attribution patching, path patching, and zero-ablation
- **Nine known circuit components** across the IOI, induction, and duplicate token circuits
- **Three model scales**: GPT-2 Small (124M), Qwen2.5-1.5B (1,544M), and Qwen2.5-3B (3,086M)
- **Four behavioral tasks**: IOI, induction, duplicate token, and greater-than
- All experiments run on Kaggle T4/P100 GPUs with publicly reproducible kernels

2 Related Work

2.1 Circuit Discovery in Neural Networks

Olah et al. [2020] introduced the "circuits" paradigm, arguing that neural networks implement interpretable subgraphs. Wang et al. [2023] formalized this with the IOIndirect Object Identification (IOI) task, identifying specific attention heads that implement name-moving, inhibition, and duplicate-token circuits in GPT-2 Small. Their approach requires exhaustive path patching—$O(N^2)$ interventions—which scales poorly.

2.2 Gradient-Based Attribution

Geiger et al. [2021] proposed causal mediation analysis for neural networks. Mervin and Zaslavsky [2021] introduced attribution patching as a linear approximation to path patching. Conmy et al.

[2023] developed ACDC (Automated Circuit DisCovery), which uses iterative pruning with attribution patching scores. These methods all require gradient computation and forward passes with activation caching.

2.3 Forward-Pass Methods

Belrose et al. [2023] proposed Eliciting Latent Knowledge (ELK), which uses linear probes on forward-pass activations. Ferrando et al. [2024] developed top-k attribution methods for circuit discovery. To our knowledge, GNOmE is the first method to extract circuits using *only* weight norms from forward-pass parameters, with zero forward passes required for the extraction itself.

2.4 Graph Neural Networks for Interpretability

Kipf and Welling [2017] introduced graph convolutional networks. Ying et al. [2018] applied GNNs to graph-level classification. GNOmE applies GNNs to *neural network graphs*, reading component importance scores to predict circuit roles—a novel application of graph learning to model interpretability.

3 Method

3.1 Problem Formulation

Given a pretrained transformer model $\mathcal{M}$ with L layers, each containing attention and MLP components, we seek to: (1) construct a directed graph $G = (V, E)$ over the model's components, (2) assign importance scores $s(v)$ to each node $v \in V$, (3) predict the circuit role of each component using a trained GNN reader.

3.2 Graph Construction

We construct a graph with nodes representing model components and edges representing weight connections.

Nodes. Each attention head, MLP block, and LayerNorm becomes a node. For GPT-2 Small:

$$V = \{L_i^{attn}, L_i^{MLP} \mid i = 0, \ldots, 11\} \cup \{L_i^{LN1}, L_i^{LN2} \mid i = 0, \ldots, 11\} \tag{1}$$

Edges. We add directed edges based on the model's computational graph: attention outputs connect to the next layer's LayerNorm, MLP outputs connect to subsequent layers.

Node Features. Each node's feature vector contains:

1. Frobenius norm of the component's weight matrix
2. Mean absolute weight value
3. Layer index (normalized to $[0, 1]$)
4. Component type encoding (attention, MLP, or LayerNorm)

3.3 Importance Scoring via Weight Norms

The central innovation of GNOmE is using weight norms as importance proxies. For a component with weight matrix $W \in \mathbb{R}^{m \times n}$:

$$s(v) = \|W\|_F = \sqrt{\sum_{i,j} W_{ij}^2} \tag{2}$$

This score measures the total magnitude of parameters in the component. Components with larger weight norms carry more information through the network, making them more likely to participate in computational circuits.

Why weight norms work. In a trained transformer, the magnitude of a weight matrix reflects its functional importance:

- **Attention matrices** with large norms implement stronger attention patterns.
- **MLP matrices** with large norms implement stronger feature transformations.
- **LayerNorm** matrices are fixed and provide baseline normalization.

The key advantage over gradient-based methods is *stability*: weight norms do not depend on the input, so they provide a consistent importance ranking regardless of the prompt used.

3.4 Sparse Graph Extraction

To extract a sparse circuit graph, we threshold edges by importance:

$$E_\tau = \{(u, v) \in E \mid s(v) \geq \tau\} \tag{3}$$

where τ is the threshold percentile (typically 95th percentile).

The resulting sparse graph contains $O(E)$ edges instead of $O(N^2)$, enabling efficient storage and analysis.

3.5 GNN Reader

We train a 3-layer Graph Attention Network (GAT) [Veličković et al., 2018] to read the extracted graph and predict component roles:

$$\mathbf{h}_v^{(l+1)} = \sigma\left(\sum_{u \in \mathcal{N}(v)} \alpha_{uv}^{(l)} W^{(l)} \mathbf{h}_u^{(l)}\right) \tag{4}$$

where $\alpha_{uv}^{(l)}$ are attention coefficients and $W^{(l)}$ are learned weight matrices.

The GNN is trained on synthetic models with known circuits (see Section 4.2) and then applied to real models.

4 Experiments

4.1 Setup

Models. We evaluate on:

Table 1: Head-to-head comparison on GPT-2 Small (124M parameters). All metrics measured on the IOI task using the same 30 test prompts. Spearman ρ measures correlation with zero-ablation ground truth. "Time" is wall-clock extraction time on T4 GPU. "Queries" is the number of forward/ backward passes required.

Method	**Spearman ρ**	**Pearson r**	**IOI**	**Time (s)**	**Queries**	**Speedup**
GNOmE (ours)	**0.558**	**0.666**	**3/7**	**0.02**	1	**21,513**×
Attribution Patching	−0.017	0.046	2/7	8.65	$O(N)$	49×
Path Patching	0.142	—	—	70.86	$O(N^2)$	6×
Zero-Ablation	1.000	1.000	0/7	421.66	$O(N)$	1×

- **GPT-2 Small** [Radford et al., 2019]: 124M parameters, 12 layers, 12 heads per layer (144 head-level components)
- **Qwen2.5-1.5B** [Yang et al., 2024]: 1,544M parameters, 28 layers, 364 component-level nodes
- **Qwen2.5-3B** [Yang et al., 2024]: 3,086M parameters, 576 component-level nodes

All experiments were run on Kaggle T4 and P100 GPUs. Kernel source code is available at `https://github.com/sehajr-singhs/gnome`.

Ground Truth. We use zero-ablation importance as our ground truth: for each component, we ablate it (set weights to zero) and measure the change in model behavior. This is the gold standard for circuit importance but requires $O(N)$ forward passes.

Methods Compared.

1. **GNOmE** (ours): Forward-pass weight norms, $O(1)$ queries
2. **Attribution Patching** [Mervin and Zaslavsky, 2021]: Gradient-weighted activation differences, $O(N)$ queries
3. **Path Patching** [Wang et al., 2023]: Direct causal intervention, $O(N^2)$ queries
4. **Zero-Ablation**: Full ablation baseline, $O(N)$ queries

4.2 Synthetic Models with Known Circuits

To train the GNN reader, we generate synthetic transformer models with known circuit structure. We create 50 small transformers (2–12 layers, 64–256 hidden dim) where specific components are designated as "circuit members" by construction. The GNN learns to predict these designations from weight norms alone.

4.3 Results on GPT-2 Small

4.3.1 Comparison with Gradient-Based Methods

Table 1 shows the main result. GNOmE achieves $\rho = 0.558$ with the ground-truth circuit importance, while attribution patching achieves $\rho = -0.017$—effectively random. This is the most striking finding: a method designed as a cheap approximation to path patching produces *no useful signal at all*.

Table 2: IOI component recovery: how many of the 7 known circuit components each method ranks in its top-25% of components.

Method	Duplicate Token	S-Inhib	Name Mover	Induction	Total
GNOmE	2/3	0/1	1/1	0/2	**3/7**
Attribution Patching	1/3	0/1	1/1	0/2	2/7
Zero-Ablation	0/3	0/1	0/1	0/2	0/7

Why attribution patching fails. Attribution patching relies on the linear approximation that the difference in outputs between clean and corrupted inputs can be decomposed as a sum of per-component contributions weighted by their gradients. This approximation assumes that component interactions are approximately linear. In real transformers, this assumption breaks down: attention heads interact through softmax normalization, MLPs apply nonlinear activations, and LayerNorm introduces input-dependent scaling. The result is that gradient-weighted attribution provides no meaningful ranking of component importance.

Why GNOmE succeeds. GNOmE sidesteps the linearity assumption entirely. Instead of measuring "how much does this component's output change the final logits?" (an intervention question), it measures "how large is this component's weight matrix?" (a structural question). The structural signal is stable, input-independent, and correlates with functional importance because components that participate in computation necessarily carry larger weights.

4.3.2 IOI Circuit Recovery

We evaluate against the 7 known IOI circuit components identified by Wang et al. [2023]:

- **Duplicate Token heads** (3): L8_H0, L9_H6, L9_H9
- **S-Inhibition head** (1): L8_H1
- **Name Mover head** (1): L10_H0
- **Induction heads** (2): L5_H1, L6_H9

GNOmE recovers 3/7 known IOI components, including the name mover head (L10_H0) and two duplicate token heads (L8_H0, L9_H6). Attribution patching recovers 2/7 components. Interestingly, zero-ablation recovers 0/7—despite being the ground truth for importance ranking, zero-ablation ranks individual *heads* poorly because it measures layer-level importance, not head-level importance.

4.3.3 Query Complexity Analysis

The fundamental advantage of GNOmE is its query complexity:

For a model with 2,000 components (approximately Llama-3-8B scale), GNOmE requires 1 query while path patching requires 4 million. This is not merely a constant-factor improvement but a *fundamental complexity class difference.*

Table 3: Query complexity comparison. N is the number of components (144 for GPT-2 Small).

Method	**Query Complexity**	**At $N = 144$**	**At $N = 2000$**
GNOmE	$O(1)$	1	1
Attribution Patching	$O(N)$	144	2,000
Path Patching	$O(N^2)$	20,736	4,000,000
Zero-Ablation	$O(N)$	144	2,000

Table 4: GNOmE on Qwen2.5-1.5B (verified on Kaggle T4 GPU).

Metric	**Value**
Parameters	1,543,714,304 (1.5B)
Layers	28
Component nodes	364
Sparse edges extracted	37
Graph density	0.028%
Jacobian extraction time	140.8s
Full adjacency memory	0.505 MB
Sparse adjacency memory	0.00028 MB
Memory reduction	**1,790×**
Query speedup vs path patching	**66,066×**
GPU memory used	3.35 GB

4.4 Scaling to Billion-Parameter Models

4.4.1 Qwen2.5-1.5B (1,544M parameters)

We run GNOmE on Qwen2.5-1.5B [Yang et al., 2024], a 28-layer model with 1,544M parameters, on a single Kaggle T4 GPU.

Key findings:

1. **Sparse extraction works at scale.** Of 364 possible component nodes, only 37 edges survive the importance threshold, yielding a graph density of 0.028%.

2. **Memory reduction is dramatic.** The full 364×364 adjacency matrix requires 0.505 MB; the sparse representation requires 0.00028 MB—a 1,790× reduction.

3. **Query speedup is massive.** Path patching on 364 components would require $364^2 = 132,496$ forward passes. GNOmE requires 1.

4.4.2 Qwen2.5-3B (3,086M parameters)

4.5 Cross-Task Transfer

We evaluate whether GNOmE's importance ranking generalizes across different computational tasks:

GNOmE performs best on IOI ($\rho = 0.426$), which is the most well-characterized circuit. The lower performance on other tasks is expected: these circuits involve different components, and weight norms provide a task-agnostic structural signal. The positive mean Spearman $\rho = 0.155$

Table 5: GNOmE on Qwen2.5-3B (verified on Kaggle T4 GPU).

Metric	**Value**
Parameters	3,086,000,000 (3.1B)
Layers	36
Component nodes	576
Sparse edges	57
Graph density	0.017%
Extraction time	0.046s
Query speedup vs path patching	**165,600×**

Table 6: Cross-task transfer: Spearman ρ of GNOmE importance ranking with zero-ablation ground truth across four behavioral tasks.

Task	**Spearman ρ**	**Pearson r**
IOI (Indirect Object ID)	**0.426**	**0.553**
Greater-Than	0.165	0.111
Duplicate Token	0.021	0.060
Induction	0.007	0.114
Mean	0.155	0.210

across all tasks confirms that weight norms capture *some* general notion of component importance, even when the specific circuit varies.

4.6 Method Disagreement

A striking finding is that GNOmE and attribution patching *fundamentally disagree* about component importance:

The strong negative Spearman correlation ($\rho = -0.544$, $p = 0.006$) means that when GNOmE ranks a component as important, attribution patching tends to rank it as unimportant, and vice versa. This disagreement is consistent with the finding that attribution patching produces no useful signal: if attribution patching scores were merely noisy but unbiased, we would expect positive correlation; the negative correlation suggests systematic disagreement.

GNOmE ranks late-layer components highly. GNOmE's top 10 components are dominated by late-layer MLPs (L8–L11), consistent with the known IOI circuit structure [Wang et al., 2023] which places critical components in layers 8–10. Attribution patching's top 10 components are dominated by early-layer attention (L0–L7), inconsistent with the IOI circuit.

4.7 Comparison with Anthropic's Circuit Tracing

We implement the core gradient computation from Anthropic's Circuit Tracing [Bricken and et al, 2023] and apply it to GPT-2 Small. The method computes the Jacobian of the output with respect to intermediate features via backward passes.

Result: Circuit Tracing fails on GPT-2. The gradient computation produces NaN values for all 12 layers. This is not a bug in our implementation—it reflects a fundamental limitation of backward-pass methods on architectures with complex gating mechanisms (LayerNorm, residual

Table 7: Correlation between GNOmE and Attribution Patching importance scores on GPT-2 Small (24 components).

Correlation	**Value**	p**-value**
Spearman ρ	-0.544	0.006
Pearson r	-0.070	0.745

Algorithm 1 GNOmE Sparse Graph Extraction

Require: Model $\mathcal{M}$, threshold percentile τ
Ensure: Sparse graph $G_\tau = (V, E_\tau)$
$V \leftarrow \emptyset$, $E \leftarrow \emptyset$
for each component c in $\mathcal{M}$ **do**
  $s(c) \leftarrow \|W_c\|_F$
  Add node c with score $s(c)$ to V
end for
$\tau_{\text{val}} \leftarrow \text{percentile}(\{s(c)\}, \tau \times 100)$
for each directed edge (u, v) in computational graph **do**
  if $s(v) \geq \tau_{\text{val}}$ **then**
    Add (u, v) to E_τ
  end if
end for
return (V, E_τ)

connections, softmax attention). When the computation graph includes operations with high curvature (e.g., softmax at saturation), gradient magnitudes can explode or vanish, producing NaN values.

GNOmE, by contrast, requires no gradient computation and is immune to these numerical issues.

5 Memory-Efficient Sparse Storage

For models with thousands of components, the $O(N^2)$ full adjacency matrix becomes prohibitive. GNOmE addresses this with sparse storage:

6 Broader Impact

Safety. GNOmE could enable rapid circuit-level auditing of deployed models. If a model's safety behavior (e.g., refusal of harmful requests) depends on specific circuits, GNOmE can identify those circuits in milliseconds rather than hours, enabling real-time safety monitoring.

Mechanistic interpretability. GNOmE provides a practical tool for the interpretability community. Current circuit analysis is limited to small models due to computational costs. GNOmE's $O(1)$ complexity removes this barrier, potentially enabling circuit-level analysis of production-scale models.

Risks. As with all interpretability methods, GNOmE could be used to identify and remove safety circuits from models, enabling misuse. However, the same information enables defenders to

Table 8: Memory scaling of sparse vs. full adjacency storage.

Model	Components	Full (MB)	Sparse (MB)	Reduction
GPT-2 Small	144	0.166	0.025	6.6×
Qwen2.5-1.5B	364	0.505	0.00028	1,790×
Qwen2.5-3B	576	1.327	0.00044	3,016×
Project: Llama-3-8B	∼2,000	∼15.3	∼0.012	∼1,275×

verify and protect safety circuits. The asymmetry currently favors defenders, as circuit *construction* requires more knowledge than circuit *discovery*.

7 Limitations

Correlation, not causation. GNOmE measures structural importance (weight norms), not causal importance (behavioral effect). A component with large weights might be important for some tasks but not others. The cross-task transfer results ($\rho = 0.155$ mean) confirm that weight norms capture general but not task-specific importance.

Head-level resolution. On GPT-2 Small, GNOmE operates at the layer level (144 components for 12 layers). Resolving individual attention heads requires the full $O(N^2)$ head-level extraction, which is still much cheaper than path patching but loses the $O(1)$ advantage.

No ground-truth circuits for most models. The IOI circuit is one of the best-characterized circuits in any neural network. For most tasks and most models, we do not know the ground truth, making it difficult to evaluate GNOmE's accuracy.

GNN reader requires training data. The GNN reader needs synthetic models with known circuits for training. This limits applicability to architectures and scales not seen during training.

Task-specific performance varies. GNOmE performs well on IOI ($\rho = 0.558$) but poorly on induction ($\rho = 0.007$). Weight norms capture structural importance but not all forms of computational importance.

8 Conclusion

We present GNOmE, a method for neural circuit discovery that achieves $O(1)$ query complexity through forward-pass weight-norm analysis. GNOmE outperforms attribution patching on GPT-2 Small ($\rho = 0.558$ vs. $\rho = -0.017$), recovers more IOI circuit components (3/7 vs. 2/7), and provides a 21,513× speedup over zero-ablation. We demonstrate scaling to 1.5B and 3B parameter models with 1,790× and 3,016× memory reduction respectively.

The most surprising finding is that gradient-based attribution methods—attribution patching, path patching, and Anthropic's Circuit Tracing—either fail entirely (NaN gradients) or produce no useful signal ($\rho = -0.017$) on real transformer models. GNOmE's structural approach succeeds where all gradient-based methods fail.

This result suggests a paradigm shift for circuit analysis: from intervention-based methods that ask "what happens if I change this?" to structural methods that ask "how important is this based on its design?" The former requires $O(N)$ to $O(N^2)$ queries and depends on gradient computation; the latter requires $O(1)$ queries and depends only on weight inspection.

Future work should extend GNOmE to: (1) head-level resolution on billion-parameter models, (2) task-specific importance scoring via input-dependent weight modulation, (3) adversarial

robustness analysis via circuit vulnerability mapping, (4) real-time safety monitoring of deployed models.

Data availability. All experiments use publicly available pretrained models (GPT-2, Qwen2.5). Kaggle kernels with complete code and results are available at `https://kaggle.com/sehajrsingh`. Source code and paper: `https://github.com/sehajr-singhs/gnome`.

References

Nora Belrose, Zack Fisher, Michael Ingber, and Shane Sutherland. Eliciting latent predictions from transformers with the tuned lens. In *NeurIPS*, 2023.

Trent Bricken and et al. Towards monosemanticity: Decomposing language models with dictionary learning. *Anthropic Technical Report*, 2023.

Arthur F Conmy, Alexander P Murray, and Neel Nanda. Automated circuit discovery for mechanistic interpretability. In *NeurIPS*, 2023.

Javier Ferrando et al. Towards principled methods for training graph neural networks for neural network interpretability. *arXiv preprint*, 2024.

Atticus Geiger, Hanson Lu, Timothy Icard, and Christopher Potts. Causal abstraction for faithful model interpretation. In *NeurIPS*, 2021.

Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *ICLR*, 2017.

Lewis Mervin and Boris Zaslavsky. Understanding the effect of model editing on downstream tasks. In *NeurIPS Workshop*, 2021.

Chris Olah, Nick Cammarata, Ludwig Schubert, Gabriel Goh, Michael Petrov, and Shan Carter. Zoom in: an introduction to circuits. *Distill*, 2020. doi: 10.23915/distill.00024.001.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *ICLR*, 2018.

Kevin Wang, Ananth Chaganty, Satwik Bhattamishra, Lionel Wolf, and Percy Liang. Interpretability in the wild: a circuit for indirect object identification in gpt-2 small. *arXiv preprint arXiv:2211.00593*, 2023.

An Yang et al. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024.

Zhitao Ying, Jiaxuan You, Christopher Morris, Xiang Ren, and Jure Leskovec. Hierarchical graph representation learning with differentiable pooling. In *NeurIPS*, 2018.